

EcoMeter – Energy Usage Monitor

Case Study 45: Backend Development · Project Documentation

Name: Mahek Yadav

Roll no: 150096725142

Cohort: Sam Altman

GitHub: <https://github.com/mahek-yadav/eco-meter>

Live frontend: <https://ecometer-energyusagemonitor.netlify.app>

Live API: <https://eco-meter-backend.onrender.com>

1. Introduction

EcoMeter is a full-stack home energy monitoring application. The core of the project is a backend REST API built with Node.js, Express.js and MongoDB. It lets users register their appliances, log electricity usage, see estimated and predicted bills, receive energy-saving tips, and get alerted when usage crosses a threshold. A React frontend is included to demonstrate the API end to end.

The backend uses Socket.io for real-time updates and Firebase for authentication and push notifications, as the case study requires.

2. Problem Statement

Develop the backend API for "EcoMeter" where:

- Users can log usage and get tips.
- Bills can be predicted and devices controlled.
- Socket.io provides real-time usage updates.
- Firebase push notifications deliver usage alerts.

Required stack: Node.js + Express.js (modular routes and controllers), MongoDB with Mongoose, JWT + Firebase authentication, Socket.io, and middleware for JWT authentication, role-based authorization and validation.

3. Requirement Coverage

Requirement	Status	Where it is implemented
Node.js + Express REST API, modular routes and controllers	Done	<code>backend/routes/</code> , <code>backend/controllers/</code>
MongoDB + Mongoose for readings, devices, users	Done	<code>backend/models/</code> (plus Tip and Alert models)
JWT authentication	Done	<code>authController.js</code> , <code>authMiddleware.js</code>
Firebase Auth	Done	<code>POST /api/auth/firebase-login</code> ; Firebase ID tokens also accepted by the <code>protect</code> middleware
Socket.io real-time usage	Done	<code>server.js</code> , <code>readingController.js</code> , <code>deviceController.js</code> , <code>alertController.js</code>
Auth, Readings, Devices, Bills, Tips, Alerts, Notifications endpoints	Done	All listed endpoints implemented (see Section 8)
Middleware: JWT auth, role-based authorization, validation	Done	<code>authMiddleware.js</code> (<code>protect</code> , <code>authorize</code>), <code>validate.js</code> (required)
Firebase push notifications for usage alerts	Done	<code>alertController.js</code> , <code>notificationController.js</code> , frontend FCM service worker
Alerts API (optional)	Done	<code>GET /api/alerts</code> , <code>POST /api/alerts/check</code>
Solar simulation API (optional)	Not implemented	Listed under future scope
API documentation (Swagger/Postman)	Done (Postman)	<code>backend/postman_collection.json</code> , <code>backend/API_ENDPOINTS.md</code>
GitHub repository with README and setup instructions	Done	Root <code>README.md</code> , <code>backend/README.md</code> , <code>frontend/README.md</code>
Bonus: deployment on Render/Heroku	Done	Backend on Render, frontend on Netlify, database on MongoDB Atlas

4. Technology Stack

Backend: Node.js (18+), Express.js 4, Mongoose 8, jsonwebtoken, bcryptjs, cors, dotenv, Socket.io 4, Firebase Admin SDK.

Database: MongoDB (local Community Server for development, MongoDB Atlas in production).

Frontend: React 18, Vite, React Router 6, Axios, Socket.io Client, Firebase Web SDK, Lucide React icons.

Deployment: Render (API), Netlify (frontend), MongoDB Atlas (database).

Tools: Postman for API testing, Git/GitHub for version control, nodemon for development.

5. System Architecture

The system has three layers that talk to each other:

1. **Client (React app):** Calls the REST API through an Axios instance that attaches the JWT as a `Bearer` token on every request. It also opens a Socket.io connection for live updates and registers a Firebase Cloud Messaging (FCM) token for push notifications.
2. **Server (Express + Socket.io):** One HTTP server hosts both the REST routes and the Socket.io server on the same port. Requests pass through CORS and JSON parsing, then the route's middleware (`protect`, `authorize`, `required`), and finally a controller. Controllers read and write MongoDB through Mongoose models.
3. **External services:** MongoDB Atlas stores all data. Firebase Admin verifies Firebase ID tokens and sends push notifications through FCM.

Typical request flow (logging a reading):

1. The client sends `POST /api/readings` with a JWT.
2. `protect` verifies the token, then `required` checks that `device` and `energyKwh` are present.
3. The controller confirms the device belongs to the user, saves the reading, and emits Socket.io events.
4. The client receives the `usageUpdate` event and refreshes the dashboard without a page reload.

The Socket.io instance is attached to the Express app with `app.set("io", io)`, so any controller can emit events through `req.app.get("io")`.

6. Database Design

MongoDB is accessed through Mongoose. All schemas use `timestamps: true`, so `createdAt` and `updatedAt` are added automatically.

7. API Reference

Base URL: `http://localhost:4000/api` locally, or `https://eco-meter-backend.onrender.com/api` in production. All routes except register, login and firebase-login need the header `Authorization: Bearer <token>`.

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Server and database status
POST	/api/auth/register	Public	Create an account and receive a JWT
POST	/api/auth/login	Public	Log in and receive a JWT
POST	/api/auth/firebase-login	Public (Firebase ID token)	Exchange a Firebase ID token for an EcoMeter JWT
POST	/api/readings	User	Log an energy reading
GET	/api/readings	User	List readings, optional from and to date filters
GET	/api/readings/	User	Readings for one device
GET	/api/devices	User	List the user's devices
GET	/api/devices/:id	User	Get one device
POST	/api/devices	User	Create a device
PUT	/api/devices/:id	User	Update a device
DELETE	/api/devices/:id	User	Delete a device
POST	/api/devices/:id/control	User	Turn a device on or off
GET	/api/bills	User	Total usage and estimated bill
GET	/api/bills/predict	User	Predicted future bill
GET	/api/bills/monthly	User	Bill for a given month

Method	Endpoint	Access	Purpose
GET	/api/tips	User	General tips plus personalised recommendations
POST	/api/tips	Admin	Add a tip
GET	/api/alerts	User	List the user's alerts
POST	/api/alerts/check	User	Check a reading against a threshold
POST	/api/notifications/register-token	User	Save the browser's FCM token
POST	/api/notifications/send	User	Send a push notification

Every resource is scoped to the logged-in user: controllers always query with `user: req.user.id`, so one user can never read or change another user's devices, readings or alerts.

8.1 Authentication

Register – POST /api/auth/register

```
{ "name": "Mahek", "email": "mahek@example.com",
  "password": "123456" }
```

Returns 201 with a message, a token and the user object. Returns 409 if the email already exists. The role is always set to `user` on registration, even if a role is sent in the request, so nobody can make themselves an admin through the API.

Login – POST /api/auth/login with `email` and `password`. Returns a token valid for 7 days, or 401 for invalid credentials.

Firestore login – POST /api/auth/firebase-login with { `"idToken": "..."` }. The server verifies the token with Firebase Admin, finds or creates the matching user (by `firebaseUid`, then by email) and returns its own JWT. Returns 503 if Firebase Admin is not configured.

8.2 Devices

POST /api/devices requires `name` and `type`:

```
{ "name": "Air Conditioner", "type": "AC", "room":  
  "Bedroom", "powerRatingWatts": 1500 }
```

PUT and DELETE only affect a device owned by the caller and return 404 otherwise.

POST /api/devices/:id/control takes { "status": "on" } or "off", rejects other values with 400, updates the device, and emits the deviceStatusUpdate Socket.io event to that user.

8.3 Readings

POST /api/readings requires device and energyKwh; voltage, current and recordedAt are optional.

```
{ "device": "DEVICE_ID", "energyKwh": 2.5, "voltage": 230,  
  "current": 6.5 }
```

The controller first checks that the device belongs to the user (404 if not). After saving, it emits usageUpdate to the user's room and liveUsage to connected clients. GET /api/readings returns readings newest first with device details populated.

8.4 Bills

All bill endpoints accept an optional ratePerKwh query parameter (default 8 per kWh).

- **GET /api/bills** adds up all of the user's readings and returns totalReadings, totalKwh, ratePerKwh and estimatedBill.
- **GET /api/bills/predict?days=30&ratePerKwh=8** predicts the bill for the next days days (default 30). Returns 400 if there are no readings.
- **GET /api/bills/monthly?year=2026&month=9** totals the readings that fall inside the chosen calendar month. Returns 400 if month is not between 1 and 12.

8.5 Tips

GET /api/tips returns three things: the general tips stored in the database, a usage summary (readingsCount, averageKwh), and a list of personalised recommendations. POST /api/tips is admin only and requires title and description.

8.6 Alerts and Notifications

POST /api/alerts/check takes { "readingId": "...", "thresholdKwh": 2 }. If the reading is within the threshold it returns alertTriggered: false. If the reading is above it, the server creates an Alert, emits usageAlert over Socket.io, and sends an FCM push notification when the user has a saved token. The response includes alertTriggered and pushNotificationSent.

POST /api/notifications/register-token saves the browser's FCM token on the user. POST /api/notifications/send sends a test notification and needs token, title and body.

The full set of ready-to-run requests is in backend/postman_collection.json.

9. Business Logic

9.1 Bill prediction

The prediction uses the average daily consumption observed so far:

- observedDays = days between the oldest and newest reading (minimum 1)
- averageDailyKwh = total kwh ÷ observedDays
- predictedKwh = averageDailyKwh × requested days
- predictedBill = predictedKwh × ratePerKwh

9.2 Personalised recommendations

The tips endpoint looks at the user's **10 most recent readings**, calculates the average kWh, and picks recommendations by usage level:

Average usage	Recommendations
No readings	Prompt to start logging usage
Up to 1 kWh (low)	Praise for efficiency, use natural light
Above 1 and up to 3 kWh (medium)	Reduce standby power, switch to LED lighting
Above 3 kWh (high)	Review high-consumption devices, optimise cooling, monitor heavy devices

General tips can be preloaded with node seedTips.js and extended by admins.

9.3 Usage alerts

An alert is raised when a reading's `energyKwh` is greater than the threshold given in the request. Alerts are stored so the user can review them later. If FCM reports that a saved token is no longer valid, the server clears it so a fresh one can be registered.

10. Authentication and Authorization

JWT: On login or registration the server signs a token containing the user `id` and `role`, valid for 7 days, using the `JWT_SECRET` environment variable. Passwords are hashed with `bcrypt` (10 salt rounds) before storage.

Firestore: The `protect` middleware first tries to verify the bearer token as an EcoMeter JWT. If that fails and Firestore Admin is configured, it tries to verify it as a Firestore ID token. This means both kinds of token work on every protected route.

Role-based access: `authorize("admin")` blocks everyone except admins with `403 Access denied`. It is applied to `POST /api/tips`. To create an admin, change a user's `role` to `admin` directly in MongoDB.

Error responses: `401` for a missing or invalid token, `403` for insufficient role, `400` for validation errors, `404` for missing resources, `409` for duplicates

11. Middleware

Middleware	File	Purpose
<code>protect</code>	<code>authMiddleware.js</code>	Verifies the JWT or Firestore ID token and sets <code>req.user</code>
<code>authorize(...roles)</code>	<code>authMiddleware.js</code>	Allows only the listed roles
<code>required(fields)</code>	<code>validate.js</code>	Returns <code>400</code> with the missing field names if any required body field is empty
CORS	<code>server.js</code>	Restricts browser access to the configured <code>CLIENT_URL</code>
<code>express.json()</code>	<code>server.js</code>	Parses JSON request bodies
404 handler	<code>server.js</code>	Returns <code>Route not found</code> for unknown routes
Error handler	<code>server.js</code>	Central handler returning a JSON error with the right status

12. Real-Time Communication (Socket.io)

Socket.io runs on the same server and port as the REST API. A client joins a private room named `user:<userId>` by emitting `joinUserRoom`, so updates for one user are not sent to others.

Event	Direction	Triggered when
<code>joinUserRoom</code>	Client → Server	The client wants its private updates
<code>joinedRoom</code>	Server → Client	Room join is confirmed
<code>usageUpdate</code>	Server → user room	A new reading is logged
<code>liveUsage</code>	Server → all clients	A new reading is logged
<code>deviceStatusUpdate</code>	Server → user room	A device is switched on or off
<code>usageAlert</code>	Server → user room	A usage alert is created
<code>deviceControl / deviceControlUpdate</code>	Client → Server → all clients	Device control relayed between connected clients

In the frontend, the dashboard, devices, readings and alerts pages subscribe to these events, so a change made in one browser tab appears in another without a refresh.

13. Firebase Push Notifications

1. The frontend asks the browser for notification permission.
2. It registers `firebase-messaging-sw.js` and obtains an FCM token using the project's VAPID key.
3. The token is sent to `POST /api/notifications/register-token` and saved on the user.
4. When an alert triggers, the backend sends a notification (title *EcoMeter Usage Alert*, with the device name, kWh used and threshold) through Firebase Admin.
5. The browser shows the message as a foreground notification, or through the service worker when the tab is in the background.

17. Testing

The API was tested with Postman using the provided collection. Recommended order:

1. Register a user, then log in and copy the token.
2. Create a device and copy its `_id`.
3. Create a reading for that device, then fetch readings and readings by device.
4. Call bills, predict and monthly.
5. Fetch tips. To test the admin route, promote the user to `admin` in MongoDB and add a tip.
6. Check an alert using a reading `_id` and a low threshold.
7. Control a device and watch the Socket.io event arrive in a second browser tab.
8. Register an FCM token and send a notification.

18. Example User Flow

Following the flow in the case study:

1. **POST** `/api/readings` logs usage. The server saves it and emits a Socket.io update, so the dashboard changes instantly.
2. **GET** `/api/readings` returns the readings, where the user can spot peak usage.
3. **GET** `/api/tips` returns saved tips plus recommendations based on the user's recent average.

19. Security Considerations

- Passwords are hashed with bcrypt and never returned in responses.
- Secrets (`JWT_SECRET`, Firebase service account, database URI) live in environment variables, and `.env` files are excluded through `.gitignore`.
- Every query is scoped to the authenticated user, which prevents access to other users' data.
- Role escalation through registration is blocked by forcing the `user` role.
- CORS limits browser access to the configured frontend URL.

20. Limitations and Future Scope

- **Solar simulation API** (optional in the case study) is not yet implemented.
- **Swagger/OpenAPI** documentation can be added alongside the Postman collection.
- **Automatic alerts:** alerts are currently checked on request. They could be triggered automatically each time a reading is logged, with per-user saved thresholds.
- **Validation:** the `required` middleware checks for missing fields only. A schema validation library such as Joi or express-validator would add type and range checks.
- **Hardening:** rate limiting, Helmet security headers and automated tests (Jest/Supertest) would make the API more production-ready.
- **Analytics:** daily and weekly charts, per-device cost breakdown and tariff slabs for more accurate bill predictions.

21. Conclusion

EcoMeter delivers every required feature of Case Study 45: a modular Node.js and Express REST API with MongoDB persistence, JWT and Firebase authentication, role-based access, real-time updates through Socket.io, and Firebase push notifications for usage alerts. It also includes the optional Alerts API, a Postman collection, a React frontend, and a live deployment on Render and Netlify.